

# Competição de Machine Learning

## Previsão de Preços de Imóveis

### 1. Visão Geral do Projeto

O objetivo deste projeto é construir um modelo preditivo para estimar o preço de venda de imóveis. Vocês atuarão como Cientistas de Dados em uma competição de modelagem baseada no famoso dataset *House Prices*. Os dados representam imóveis residenciais localizados em Ames, Iowa (EUA), e contam com 79 variáveis explicativas que descrevem quase todos os aspectos das casas — desde atributos básicos como área total, bairro e ano de construção, até detalhes minuciosos como o material do telhado, a qualidade da cozinha e a altura do teto do porão.

Partindo da análise exploratória detalhada desses atributos até a entrega de um modelo preditivo automatizado, a avaliação final considerará a capacidade do modelo de generalizar para dados inéditos, a organização do código e a capacidade analítica da equipe em extrair valor desses dados.

### 2. Materiais Fornecidos

Todo o material base para o desenvolvimento do projeto está dividido entre os arquivos de dados e o repositório de template.

#### Arquivos de Dados:

Disponíveis no diretório do projeto em: [\[LINK\]](#)

- **treino.csv**: Conjunto de dados contendo as variáveis explicativas e a variável alvo (**SalePrice**), destinado ao treinamento e validação local dos modelos.
- **teste\_publico.csv**: Amostra de dados sem a variável alvo, fornecida exclusivamente para que a equipe teste se o script de predição (**pipeline.py**) consegue ler um arquivo novo e processá-lo sem erros.
- **descricao\_dados.txt**: Dicionário de dados detalhando o significado de cada variável e suas respectivas categorias.

**Nota:** A base **teste\_secreto.csv** não será disponibilizada aos alunos. Ela é mantida sob posse exclusiva do professor e será processada pelo pipeline da equipe apenas na etapa de avaliação automatizada para calcular o ranking final.

#### Repositório Template:

- **Link:** [\[LINK\]](#)
- **O que é:** Um repositório base no GitHub criado pelo professor contendo a estrutura inicial exigida para a entrega final. Ele traz o esqueleto dos arquivos que serão avaliados pelo corretor automático. As equipes devem utilizá-lo como ponto de partida (via *fork* ou clone) para construir o seu Repositório de Submissão.

### 3. Estrutura de Entrega

A entrega final será feita através do envio do **Relatório Técnico em PDF** na atividade aberta no AVA, até a data limite estipulada. O relatório é o documento central e deve conter, logo na capa, os **links públicos para os dois repositórios no GitHub** detalhados abaixo:

#### A. Repositório de Submissão

Repositório clonado/feito a partir do template do professor, focado apenas na execução automatizada. Deve conter:

1. `pipeline.py`: Script com a função de predição. Todos os tratamentos (imputação, escalonamento, encoding) devem estar integrados neste script (recomenda-se o uso de Pipeline do Scikit-Learn). **Atenção ao formato de saída:** O script deve retornar *exclusivamente* um array do Numpy (`np.array`), uma lista ou uma `pd.Series` contendo apenas os valores contínuos das predições (sem a coluna de ID), rigorosamente na mesma ordem das linhas do arquivo de teste.
2. `modelo.pkl` (ou `.joblib`): O modelo treinado e salvo.
3. `requirements.txt`: Dependências necessárias para a execução.

**Atenção:** O código deve rodar de forma 100% autônoma. Se o script falhar ao receber o CSV de teste do professor ou retornar os dados em um formato diferente do exigido, a equipe receberá nota zero no critério de Ranking.

**Tempo Limite:** O script `pipeline.py` deve processar a base de teste e retornar as predições em um tempo máximo de 1 minuto na máquina de avaliação.

#### B. Repositório de Desenvolvimento

Repositório contendo os materiais de trabalho da equipe:

- Notebooks (`.ipynb`) com a Análise Exploratória (EDA).
- Notebooks de pré-processamento, testes de algoritmos e otimização.

---

### 4. Escopo do Relatório Técnico (PDF)

O relatório técnico é a principal vitrine do raciocínio da equipe. Ele deve seguir rigorosamente a estrutura abaixo:

1. **Capa:** Deve conter obrigatoriamente a identificação da equipe (ex: Grupo A), o nome completo de todos os integrantes e os links públicos para os dois repositórios no GitHub (Submissão e Desenvolvimento).
2. **Introdução:** Breve contexto do problema, da base de dados e definição de qual métrica a equipe focou para otimizar o modelo.
3. **Análise Exploratória de Dados (EDA):**
  - Apresentação visual (gráficos) das principais descobertas.
  - Identificação de correlações relevantes com a variável alvo.

#### 4. Pré-processamento e Feature Engineering:

- Como a equipe lidou com dados faltantes (nulos) e valores atípicos (outliers).
- Transformações aplicadas (escalonamento, tratamento de dados categóricos).
- Novas variáveis criadas (se houver) e a lógica por trás delas.

#### 5. Modelagem e Validação:

- Quais algoritmos foram testados.
- Qual técnica de validação foi usada para evitar o *overfitting* (ex: Validação Cruzada).
- Como foi feita a busca pelos melhores hiperparâmetros.

#### 6. Conclusões:

- Tabela comparativa consolidando as métricas e os modelos de Machine Learning que foram testados durante o desenvolvimento da equipe (comparando o tempo de treino e, **obrigatoriamente, o  $R^2$ , o RMSLE e o MAE em dólares** de algoritmos como Regressão Linear, Random Forest, XGBoost, etc).
- Justifique o erro do seu modelo em termos de impacto para o negócio. (Exemplo: "Nosso modelo final obteve um MAE de \$ 15.000, o que significa que ele erra o preço de avaliação das casas, em média, em 15 mil dólares para mais ou para menos").
- Justificativa final: com base nos testes e nessas métricas, por que o modelo enviado no repositório de submissão foi o escolhido?

---

## 5. Avaliação e Pontuação (0 a 10 pontos)

A nota final do projeto será a soma dos quatro critérios abaixo, priorizando o desempenho prático e o engajamento contínuo da equipe.

### 1. Ranking de Desempenho (Métrica: RMSLE) - Até 3,5 pontos

- **0,0 pontos:** Modelo apresentou erro ao rodar **OU** teve desempenho igual ou inferior ao *Baseline* (Regressão Linear simples executada pelo professor).
- **1,5 pontos** (Pontuação Mínima de Sucesso): Garantidos para qualquer equipe cujo modelo supere o desempenho do Baseline.
- **3,5 pontos:** Equipe(s) com o melhor desempenho (menor erro) no teste secreto.
- **Notas intermediárias:** O intervalo restante (de 1,5 a 3,5 pontos) será distribuído de forma proporcional ao desempenho das equipes que superarem o Baseline, premiando os modelos com maior capacidade preditiva.

### 2. Acompanhamentos em Sala de Aula - Até 3,0 pontos

Serão dois *check-ins* (1,5 pontos cada). A nota é garantida pela apresentação do progresso no prazo:

- **Acompanhamento 1 [Data disponível no AVA]:** Apresentar Análise Exploratória e estratégia de limpeza de dados.

- **Acompanhamento 2 [Data disponível no AVA]:** Apresentar a estrutura do `pipeline.py` rodando e resultados preliminares dos primeiros modelos.

### 3. Relatório Técnico - Até 2,0 pontos

- Avalia a clareza textual, a profundidade das justificativas, a qualidade das visualizações (gráficos) e o cumprimento estrito do escopo definido na Seção 4.

### 4. Repositório de Desenvolvimento - Até 1,5 pontos

- Avalia a organização dos notebooks, uso de comentários explicativos no código e garantia de reprodutibilidade dos experimentos (uso de *random seeds*, lógica clara).

---

## 6. Dicas e Orientações Gerais

- **Atenção à Métrica:** A métrica da competição (RMSLE) penaliza erros em proporção. Porém, modelos tradicionais (como Regressão Linear ou Árvores) minimizam o erro absoluto (MSE). *Dica:* Aplique o logaritmo na variável alvo antes de treinar o modelo (ex: `y_treino = np.log1p(SalePrice)`). Isso força o algoritmo a otimizar a proporção do erro. **Mas cuidado extremo:** no seu `pipeline.py`, as previsões finais devem ser obrigatoriamente convertidas de volta para dólares usando a função exponencial (ex: `np.expm1(predicoes)`) antes de serem retornadas, senão seu erro na avaliação será gigantesco! (*Pesquise sobre a classe `TransformedTargetRegressor` do Scikit-Learn para automatizar isso.*)
- **Leia o Dicionário de Dados:** Antes de apagar colunas inteiras por terem muitos valores "nulos", verifique se o valor vazio não possui um significado próprio no contexto do negócio (ex: "NaN" na coluna Piscina significa que a casa não tem piscina, e não um dado perdido).
- **Cuidado com o Data Leakage (Vazamento de Dados):** O modelo nunca deve ter acesso a informações do teste durante o treinamento. Estatísticas de escalonamento (como o `StandardScaler`) ou padrões de *Target Encoding* devem ser calculados *apenas* nos dados de treino.
- **Automatize com Pipelines:** O uso das classes `Pipeline` e `ColumnTransformer` do Scikit-Learn facilita muito a exportação do modelo final e garante que novos dados passem exatamente pelas mesmas transformações do treino.
- **Comece Simples:** Não pule direto para algoritmos complexos (como *ensembles* pesados ou redes neurais). Crie um modelo simples primeiro para ter a sua própria base de comparação (*baseline* interno).
- Configurar os encoders (como o `OneHotEncoder`) com o parâmetro `handle_unknown='ignore'` para evitar que o código quebre caso a base de teste possua categorias que não existiam no treino."